Atlas

Parametric flood protection for Houston

Fast cash in the first 72 hours, triggered by public gauge data, settled on Solana, priced on Tiger Data

---

20-Hour Build Plan

CodeRed Hackathon | Tracks: Tiger Data, Solana, Overall

Team: 2 builders, JavaScript/TypeScript and Python only (no Rust)

The one-sentence pitch

When a Houston bayou crosses flood stage, Atlas pays every covered household near that gauge a fixed sum within minutes, with the trigger read from an on-chain oracle, policies issued as Token-2022 tokens, payouts governed by a Squads multisig, and catastrophe losses absorbed by investors holding an on-chain catastrophe bond.

Assumptions behind this plan: a team of two, a 20-hour clock, devnet only, no custom Solana program. Items marked **[VERIFY]** are things the plan depends on that have not been confirmed and must be checked in Hour 0 to 1.

Contents

1 Product summary and scope **3**
1.1 The problem . . . 3
1.2 What Atlas does . . . 3
1.3 How each sponsor technology is used . . . 3
1.4 In scope for the 20 hours . . . 3
1.5 Explicitly out of scope . . . 3

2 System architecture **4**
2.1 Data flow for a single payout . . . 4
2.2 Tech stack . . . 5

3 Pre-flight checklist (before Hour 0) **5**

4 Team roles **5**

5 Hour-by-hour timeline **6**

6 Data layer: Tiger Data **7**
6.1 Sources . . . 7
6.2 Schema . . . 7
6.3 Continuous aggregates, compression, retention . . . 8
6.4 Trigger query . . . 9
6.5 Replay engine . . . 9

7 Pricing and capital model **9**
7.1 Event frequency . . . 9
7.2 Premium . . . 10
7.3 Uncertainty, shown on screen . . . 10
7.4 Correlated catastrophe and the bond . . . 10
7.5 Underwriting rules (anti-adverse-selection) . . . 10

8 Solana layer **11**
8.1 Policies: Token-2022 . . . 11
8.2 Oracle: Switchboard on-demand . . . 11
8.3 Pool: Squads multisig . . . 11

8.4 Catastrophe bond token . . . 11
8.5 Blink purchase flow . . . 12
8.6 Evidence anchoring . . . 12

9 Backend and frontend **12**
9.1 API (FastAPI) . . . 12
9.2 Pages . . . 12

10 Demo script (4 minutes) **13**

11 Judge Q&A preparation **14**

12 Risk register and fallbacks **14**

13 Limitations of the product **15**
13.1 Insurance design . . . 15
13.2 Data and oracle . . . 15
13.3 Pricing and capital . . . 16
13.4 Legal and regulatory . . . 16
13.5 Users and adoption . . . 16
13.6 Hackathon build . . . 16

14 Submission checklist **16**

15 Unverified assumptions to check in Hour 0 to 1 **17**

1 Product summary and scope

1.1 The problem

Flood insurance in the US runs mostly through the federal program, which has a waiting period before new policies take effect and claims that take weeks or months to pay. Many Houston renters and homeowners have no flood coverage at all. In the first 72 hours after a flood, people need money for a hotel, a rental car, food, and lost wages, and nothing pays that fast.

1.2 What Atlas does

Atlas sells a small policy tied to the nearest stream gauge. If that gauge crosses a defined flood level during the policy term, the policyholder receives a fixed payout (for example $500) automatically. There is no claim form and no adjuster. Atlas does *not* pay for property damage and is not a replacement for flood insurance. It covers immediate needs.

1.3 How each sponsor technology is used

Tiger Data	Decades of gauge history and 15-minute live readings as hypertables; continuous aggregates for trigger detection; burn analysis and pricing; catastrophe stress tests; compression and retention policies; the Harvey replay engine.
Solana	Policies as non-transferable Token-2022 tokens with on-chain metadata and a permanent delegate; a Switchboard on-demand oracle feed carrying the gauge reading on-chain; a Squads multisig vault with a spending limit for the payout bot; an SPL catastrophe bond token for capital providers; Blinks for one-tap purchase; Memo anchoring of trigger evidence.

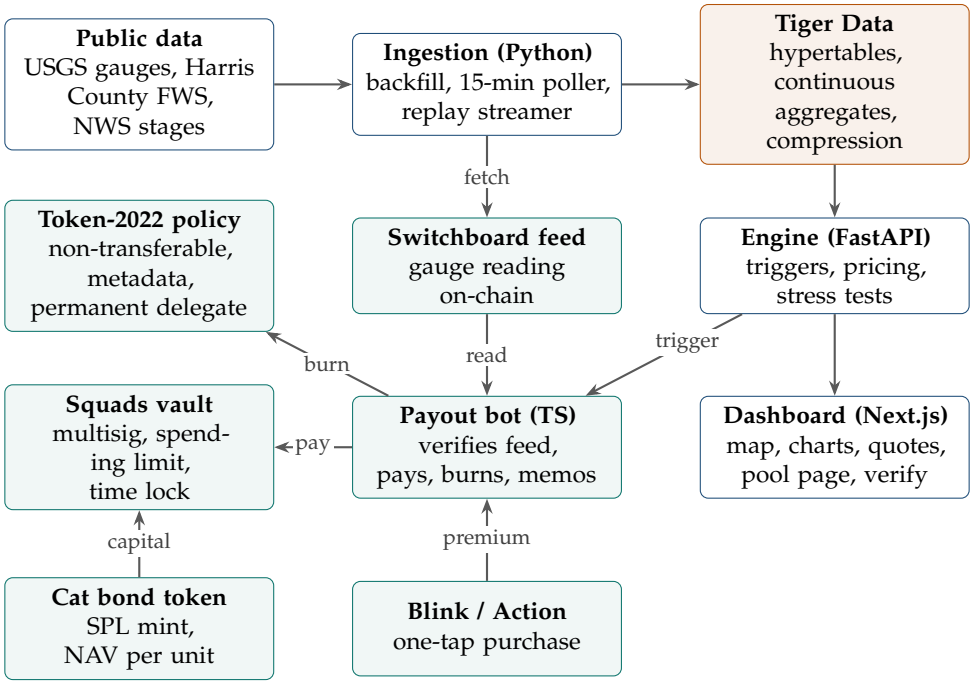
1.4 In scope for the 20 hours

- 6 to 10 Houston-area gauges with historical and live data.
- Pricing engine with confidence intervals.
- Policy purchase, payout on trigger, one payout per policy term.
- Catastrophe bond token with NAV accounting.
- Harvey replay through the full pipeline.
- Dashboard: gauge map, live chart, quote page, pool page, verify button.

1.5 Explicitly out of scope

Mainnet, real money, a custom Solana program, KYC, a mobile app, rainfall triggers (stretch only), property-level flood modeling, and any claim of being a licensed insurance product.

2 System architecture



2.1 Data flow for a single payout

1. The poller writes a new gauge reading into Tiger Data every 15 minutes.
2. The trigger query (Section 6.4) detects that a gauge has crossed its threshold.
3. The Switchboard feed for that gauge is updated so the reading exists on-chain.
4. The payout bot reads the on-chain feed value and confirms it is at or above the policy threshold. It does not trust the database alone.
5. The bot selects active policies on that gauge that are past their waiting period and have not already been paid this term.
6. Payouts execute from the Squads vault through the bot's spending limit.
7. A Memo transaction records a hash of the trigger evidence (gauge ID, timestamp, reading, feed account, slot).
8. Each paid policy is burned through the permanent delegate, enforcing one payout per term.
9. The payout is logged in Tiger Data and appears on the pool page.

2.2 Tech stack

Database	Tiger Cloud (Tiger Data). Fallback: `timescale/timescaledb` Docker image locally.
Data and engine	Python 3.11, pandas, numpy, scipy, psycopg, FastAPI, httpx.
Chain	TypeScript, `@solana/web3.js`, `@solana/spl-token` (Token-2022), `@sqds/multisig`, `@switchboard-xyz/on-demand`, `@solana/actions`. **[VERIFY]** package names and current APIs.
Frontend	Next.js, Tailwind, a charting library (Recharts), Leaflet for the gauge map, Solana wallet adapter.
RPC	A free-tier Solana RPC provider key for devnet (public devnet endpoints rate-limit hard).
Wallets	Three Phantom or Solflare wallets on devnet for demo policyholders, one for the investor, plus the bot keypair and multisig members.
Repo	One monorepo: `/data`, `/engine`, `/chain`, `/web`, `/docs`.

3 Pre-flight checklist (before Hour 0)

Do these before the clock starts if the rules allow, otherwise in the first 30 minutes.

- ☐ Tiger Cloud account created, service running, connection string tested from both laptops. Note free-tier limits. **[VERIFY]**
- ☐ RPC provider key for devnet.
- ☐ Devnet SOL in every wallet. Airdrops get rate-limited, so fund early and from several sources.
- ☐ Test stablecoin decided: Circle's devnet USDC faucet **[VERIFY]**, otherwise your own SPL mint named "aUSD (test)".
- ☐ GitHub monorepo with the folder layout above, `.env.example`, and a shared secrets file outside the repo.
- ☐ Node 20+, Python 3.11, Solana CLI installed on both laptops.
- ☐ Gauge shortlist drafted (Section 6.1).
- ☐ Track descriptions and judging criteria read and pasted into `/docs/criteria.md`.

4 Team roles

Person A: Data and risk	Tiger Data schema, ingestion, trigger logic, pricing, stress tests, catastrophe bond math, replay engine, FastAPI. Owns every number shown on stage. (Suggested: the teammate with actuarial and statistics training.)
Person B: Chain and product	Token-2022 policies, Switchboard feed, Squads vault, payout bot, Blink, catastrophe bond token, Next.js dashboard. Owns the live demo flow.
Shared	Integration at each gate, the pitch, Q&A preparation, the recorded fallback.

Rule for the whole build

If a task blocks you for more than 45 minutes, say so out loud and switch to its fallback (Section 12). The project fails by quietly sinking three hours into one SDK, not by having a fallback in the demo.

5 Hour-by-hour timeline

Hours	Person A (data and risk)	Person B (chain and product)
0 to 1	Confirm USGS API access and history depth for 2 gauges. Confirm NWS flood stages for them. Create Tiger schema.	Confirm Token-2022 extension combination mints on devnet. Create a Squads multisig on devnet. Attempt a Switchboard feed creation.
Gate G1 (Hour 1): each [VERIFY] item is green, or its fallback is chosen and written in `/docs/decisions.md`.		
1 to 4	Backfill: full instantaneous history for the Harvey window (Aug 20 to Sep 10, 2017) and annual peaks for all years available. Start the live 15-minute poller.	Policy mint script: create a policy with metadata (gauge, threshold, payout, term, waiting period). Squads vault funded with test stablecoin. Spending limit configured for the bot.
4 to 7	Continuous aggregates, trigger query with declustering, burn analysis, premium with bootstrap confidence interval.	Switchboard feed reading one gauge (or fallback). Payout bot: read feed, select policies, pay through spending limit, Memo evidence, burn policy.
Gate G2 (Hour 7): one test policy on one gauge goes from trigger to payout end to end on devnet. If not, cut scope now.		
7 to 10	FastAPI: `/quote`, `/gauges`, `/policies`, `/pool`, `/events`. Harvey stress test and catastrophe bond sizing.	Blink for purchase (quote → pay premium → mint policy). Catastrophe bond token: mint on deposit, NAV per unit, redemption.
10 to 11	Integration, eat, re-check every service is still running. Commit and tag `v0.1`.	
11 to 14	Replay engine: stream Harvey readings through the same pipeline at 200x speed, with a feed pointed at the replay endpoint.	Dashboard: gauge map, live chart with threshold line, quote page, pool page (premiums, payouts, reserves, bond NAV), verify button.
14 to 15	Staggered rest: A rests 14:00 to 15:00 while B polishes UI; B rests 15:00 to 16:00 while A runs checks.	
15 to 16	Numbers audit: every figure on screen reproduced by a query in `/docs/numbers.sql`.	(Resting until 16)
Gate G3 (Hour 16): feature freeze. Only bug fixes and demo work after this point.		

Hours	Person A (data and risk)	Person B (chain and product)
16 to 18	Pitch slides with the pricing and stress-test numbers. Q&A rehearsal.	Two full demo rehearsals. Record the fallback video of the complete Harvey replay.
18 to 19	README, architecture diagram, setup steps.	Submission page, screenshots, track tags.
19 to 20	Buffer. Submit by 19:30. Final rehearsal.	

What "done" means at each gate

G1: every unknown resolved or swapped for a fallback. **G2:** a real devnet transaction pays a real test policy because a real gauge reading crossed a threshold. **G3:** the full Harvey replay runs start to finish twice without touching the code.

6 Data layer: Tiger Data

6.1 Sources

- **USGS Water Services:** instantaneous stage and discharge, typically every 15 minutes, plus annual peak records going back decades for many sites. **[VERIFY]** endpoint parameters and history depth per site.
- **Harris County Flood Warning System:** dense local gauge and rainfall network. **[VERIFY]** whether a public API exists; if not, skip it.
- **NWS flood stages:** official action, minor, moderate and major flood stages per forecast point, used to set thresholds where available. **[VERIFY]** access.

Gauge selection: pick 6 to 10 gauges on major Houston bayous (for example Buffalo, Brays, White Oak, Greens, Sims, Cypress Creek) that have (a) live data, (b) at least 20 years of history, and (c) data during Harvey. Look up the actual site numbers; do not guess them. Note any gauge that went offline during Harvey, since that is evidence for the limitations section.

6.2 Schema

```
-- Raw readings: one row per gauge per timestamp
CREATE TABLE gauge_readings (
  time TIMESTAMPTZ NOT NULL,
  gauge_id TEXT NOT NULL,
  stage_ft DOUBLE PRECISION,
  discharge_cfs DOUBLE PRECISION,
  source TEXT NOT NULL, -- 'usgs', 'replay'
  ingested_at TIMESTAMPTZ DEFAULT now()
);
SELECT create_hypertable('gauge_readings', by_range('time'));
CREATE UNIQUE INDEX ON gauge_readings (gauge_id, time, source);

-- Gauge metadata (regular table)
```

```
CREATE TABLE gauges (
  gauge_id TEXT PRIMARY KEY,
  name TEXT,
  lat DOUBLE PRECISION,
  lon DOUBLE PRECISION,
  trigger_ft DOUBLE PRECISION, -- policy trigger level
  nws_flood_ft DOUBLE PRECISION,
  feed_pubkey TEXT -- Switchboard feed account
);

-- Annual peaks for pricing
CREATE TABLE annual_peaks (
  gauge_id TEXT, water_year INT, peak_stage_ft DOUBLE PRECISION,
  peak_time TIMESTAMPTZ, PRIMARY KEY (gauge_id, water_year)
);

-- Policies mirror on-chain state for fast queries
CREATE TABLE policies (
  policy_mint TEXT PRIMARY KEY,
  owner TEXT NOT NULL,
  gauge_id TEXT REFERENCES gauges,
  payout_usd NUMERIC NOT NULL,
  premium_usd NUMERIC NOT NULL,
  starts_at TIMESTAMPTZ NOT NULL, -- after waiting period
  ends_at TIMESTAMPTZ NOT NULL,
  status TEXT NOT NULL -- active, paid, expired
);

-- Events and payouts as hypertables
CREATE TABLE trigger_events (
  time TIMESTAMPTZ NOT NULL, gauge_id TEXT NOT NULL,
  reading_ft DOUBLE PRECISION, feed_slot BIGINT, evidence_hash TEXT
);
SELECT create_hypertable('trigger_events', by_range('time'));

CREATE TABLE payouts (
  time TIMESTAMPTZ NOT NULL, policy_mint TEXT, owner TEXT,
  amount_usd NUMERIC, tx_sig TEXT, memo_sig TEXT
);
SELECT create_hypertable('payouts', by_range('time'));
```

6.3 Continuous aggregates, compression, retention

```
CREATE MATERIALIZED VIEW gauge_hourly
WITH (timescaledb.continuous) AS
SELECT time_bucket('1 hour', time) AS bucket, gauge_id,
     max(stage_ft) AS max_stage, avg(stage_ft) AS avg_stage,
     max(discharge_cfs) AS max_q, count(*) AS n
FROM gauge_readings
GROUP BY bucket, gauge_id;

SELECT add_continuous_aggregate_policy('gauge_hourly',
  start_offset => INTERVAL '3 days',
  end_offset => INTERVAL '15 minutes',
  schedule_interval => INTERVAL '15 minutes');

ALTER TABLE gauge_readings SET (
  timescaledb.compress,
```

```
  timescaledb.compress_segmentby = 'gauge_id',
  timescaledb.compress_orderby = 'time DESC');
SELECT add_compression_policy('gauge_readings', INTERVAL '7 days');
```

Show the compression ratio on a slide (`hypertable_compression_stats`). It is a cheap, concrete Tiger Data talking point.

6.4 Trigger query

A gauge is in a trigger state when its most recent reading is at or above `trigger_ft`. An *event* starts at the first crossing and ends after 72 consecutive hours below the threshold, so one flood counts once.

```
SELECT g.gauge_id, r.time, r.stage_ft
FROM gauges g
JOIN LATERAL (
  SELECT time, stage_ft FROM gauge_readings
  WHERE gauge_id = g.gauge_id AND source = $1 -- 'usgs' or 'replay'
  ORDER BY time DESC LIMIT 1
) r ON true
WHERE r.stage_ft >= g.trigger_ft
  AND NOT EXISTS ( -- no open event already
    SELECT 1 FROM trigger_events e
    WHERE e.gauge_id = g.gauge_id
      AND e.time > r.time - INTERVAL '72 hours'); -- works for replay too
```

Data quality guards: ignore a reading that jumps more than a set number of feet from the previous one within 15 minutes, and require two consecutive readings above threshold before firing. Log rejected readings so you can show the guard working.

6.5 Replay engine

The replay streamer reads the stored Harvey window and re-inserts each reading with `source = 'replay'` at 200x speed (15 minutes of real time per 4.5 seconds). The trigger query, oracle feed, bot and dashboard all run unchanged against the replay source. This is what the judges watch.

7 Pricing and capital model

7.1 Event frequency

For gauge g with T_g years of record and N_g declustered events at or above its trigger level:

$$\hat{\lambda}_g = \frac{N_g}{T_g}.$$

Use annual peaks for the long record and instantaneous data to decluster recent years. Where few events exist, also fit a generalized extreme value distribution to annual peaks using scipy's GEV implementation and compare the implied exceedance probability (stretch goal).

7.2 Premium

Each policy pays a fixed benefit B at most once in a term of length t years. Treating events as Poisson:

$$P(\text{payout}) = 1 - e^{-\hat{\lambda}_g t}, \qquad \mathbb{E}[L] = B\left(1 - e^{-\hat{\lambda}_g t}\right).$$

The premium adds a loading θ for expenses and uncertainty and a capital charge:

$$\pi = \mathbb{E}[L](1 + \theta) + c \cdot k,$$

where k is the policy's share of the catastrophe capital it requires and c is the yield paid to catastrophe bond investors.

7.3 Uncertainty, shown on screen

Bootstrap over years: resample the record with replacement 2,000 times, recompute $\hat{\lambda}_g$ and π, and display the premium as a central value with a 5th to 95th percentile band. With few extreme events the band will be wide. Showing it is a strength in front of finance judges, not a weakness.

7.4 Correlated catastrophe and the bond

In a large flood many gauges trigger together. For a scenario s (Harvey replay), aggregate payout is

$$S_s = B \cdot \#\{\text{active policies on gauges triggered in } s\}.$$

Losses are absorbed in layers:

1. **Premium reserve** R**:** premiums collected, net of expenses.
2. **Catastrophe bond capital** C**:** absorbs $\max(0, S_s - R)$ up to C. Investors earn yield c from premiums in return.
3. **Above** $R + C$**:** payouts are reduced pro rata. State this openly in the product terms.

The bond token's NAV per unit is

$$\text{NAV}_t = \frac{C - \text{losses absorbed}_t + \text{yield accrued}_t}{\text{units outstanding}}.$$

Size C so the pool survives a Harvey-scale event at the demo portfolio size, and show the investor loss under that scenario.

7.5 Underwriting rules (anti-adverse-selection)

- Waiting period of 7 days between purchase and coverage start (shortened in demo mode, and said so).
- Sales close for gauges in an area under a flood watch or with a named storm within a set distance.
- One policy per wallet per gauge, and a cap per wallet.

8 Solana layer

8.1 Policies: Token-2022

Each policy is a mint of supply 1 with decimals 0 and these extensions:

- **NonTransferable:** the policy cannot be sold or moved.
- **MetadataPointer + TokenMetadata:** stores `gauge_id`, `trigger_ft`, `payout`, `starts_at`, `ends_at`, `premium`, `terms_hash`.
- **PermanentDelegate:** the protocol can burn the policy after payout or at expiry.

[VERIFY] in Hour 0 that all three combine on one mint through the JS library. Fallback: drop PermanentDelegate and track expiry in the database, or issue policies as compressed NFTs.

8.2 Oracle: Switchboard on-demand

One feed per gauge whose job fetches the latest stage from the USGS endpoint and parses the value. The bot cranks an update when the trigger query fires, then reads the on-chain value before paying. Anyone can check that the on-chain reading crossed the threshold.

For the replay, a second feed points at the replay endpoint. Say clearly in the demo that the replay feed reads your server, while the live feed reads USGS directly.

[VERIFY] custom HTTP feeds can be created and updated on devnet from TypeScript. **Fallback:** the bot signs each reading and writes it with the Memo program; present the oracle as the production design.

8.3 Pool: Squads multisig

- 2-of-3 multisig (both teammates plus a third key standing in for an independent trustee).
- Spending limit for the bot key: a capped amount per period, usable without other signers.
- Larger moves (reserve withdrawals, catastrophe-mode batch payouts) require a multisig proposal with a time lock.

[VERIFY] spending limits and time locks through the SDK on devnet. Fallback: plain multisig for reserves, a separately funded hot wallet for the bot with a small balance.

8.4 Catastrophe bond token

An SPL mint. Investors deposit test stablecoin into the vault and receive units at current NAV; redemption burns units at NAV after a notice period. Units are transferable. NAV is computed by the engine from on-chain balances and published on the pool page.

8.5 Blink purchase flow

An Action endpoint returns the quote for a chosen gauge; the POST returns a transaction that pays the premium to the vault. On confirmation, the backend mints the policy to the buyer. Test with a Blink inspector before relying on it. **[VERIFY]** current Actions spec. Fallback: a normal "Buy" button using the wallet adapter.

8.6 Evidence anchoring

Each trigger writes a Memo containing a hash of the tuple (`gauge_id`, `reading`, `time`, `feed_pubkey`, `feed_slot`). The dashboard's verify button recomputes the hash from Tiger Data and matches it to the Memo on-chain.

9 Backend and frontend

9.1 API (FastAPI)

`GET /gauges`	Gauges with latest reading, threshold, status.
`GET /gauges/{id}/series`	Hourly series from the continuous aggregate, with threshold.
`GET /quote?gauge=&payout=`	Premium, probability, confidence band, waiting period, sales status.
`GET /policies?owner=`	Policies and status.
`GET /pool`	Reserve, bond capital, NAV, premiums in, payouts out, coverage outstanding.
`GET /events`	Trigger events with evidence hashes and transaction links.
`GET /stress?scenario=harvey`	Aggregate loss, layer absorption, investor loss.
`POST /replay/start`	Starts the Harvey replay.

9.2 Pages

- **Map:** Houston gauges colored by status (normal, near, triggered).
- **Gauge view:** stage over time with the trigger line, events marked, "buy coverage" panel.
- **Pool:** reserves, bond NAV chart, payouts feed with explorer links, stress-test results.
- **Verify:** paste a payout transaction and see the matching reading, feed value, and Memo hash.

10 Demo script (4 minutes)

0:00 to 0:30	The gap: federal flood claims take weeks; people need money in the first 72 hours. One sentence on Atlas.
0:30 to 1:15	A judge names a Houston bayou. Show the quote: probability, premium, confidence band, and the 30-year history behind it. Buy a policy via Blink on a teammate's phone; show the non-transferable policy token in the wallet.
1:15 to 2:45	Start the Harvey replay. The map turns from green to red as bayous rise. Triggers fire, the oracle updates on-chain, payouts land in three wallets, policies burn. The pool page shows the reserve exhausted and the catastrophe bond absorbing the rest, with NAV dropping.
2:45 to 3:15	Press verify on one payout: reading, on-chain feed value, Memo hash all match.
3:15 to 3:45	The numbers: loss ratio, how much capital a Harvey-scale event needs, compression ratio in Tiger Data.
3:45 to 4:00	Limitations in one breath (basis risk, licensing), then the close.

Demo rules

Have the recorded fallback queued in a browser tab. Pre-fund all wallets. Run the replay once 20 minutes before judging to warm caches. Never debug in front of judges: switch to the video within 15 seconds of anything breaking.

11 Judge Q&A preparation

Question	Answer
What if the gauge triggers but my street stayed dry?	That is basis risk, the core trade-off of parametric insurance. You trade precision for speed. We limit it with nearest-gauge assignment and fixed immediate-needs payouts rather than damage estimates. Rainfall and multi-gauge triggers are the next step.
Is this legal?	Selling insurance needs a license, and the bond token could be a security. Atlas is the engine an insurer, a city program or a nonprofit would run; we would partner rather than sell directly.
Why blockchain?	Payouts in seconds to anyone with a phone wallet, a trigger anyone can verify, policies nobody can resell, and a pool whose solvency is public in real time. A traditional insurer offers none of those.
Can you fake the oracle?	The live feed fetches directly from USGS through Switchboard; we cannot alter it. The replay feed is our server and we say so. Production would use multiple sources.
What if the pool runs out?	Layered capital: premiums first, then the catastrophe bond, then pro-rata reduction. We show the Harvey stress test.
Why not buy coverage when a storm is coming?	Waiting period and sales freeze during watches.
What if gauges fail in a flood?	They sometimes do. Fallback is the nearest functioning gauge or a rainfall trigger, defined in the policy terms.
How did you price it?	Declustered event frequency from decades of gauge history, Poisson payout probability, loading plus capital charge, bootstrap confidence band.

12 Risk register and fallbacks

Risk	Odds	Switch trigger	Fallback
Switchboard custom feed will not work on devnet	Medium	Not reading a value by Hour 6	Bot-signed readings via Memo; oracle described as production path
Squads spending limits fight back	Medium	Not working by Hour 4	Plain multisig for reserves; small hot wallet for the bot
Token-2022 extension combination fails	Low-Med	Fails in Hour 0 test	Drop PermanentDelegate; or compressed NFTs
USGS history too short or gappy	Low-Med	Hour 1 check	Switch gauges; use annual peaks only for pricing
Tiger Cloud limits or outage	Low	Any blocking error	Local Docker TimescaleDB, same schema

Risk	Odds	Switch trigger	Fallback
Devnet congestion or airdrop limits	Medium	Failed airdrops	Pre-funded wallets, own test mint, retry with priority fee
Blink rendering issues	Medium	Not working by Hour 9	Buy button via wallet adapter
Replay too slow or out of order	Low	Rehearsal	Pre-computed replay timeline, same visuals
Live demo failure on stage	Medium	Anything breaks	Recorded video, 15-second rule
Running out of time	High	Missed gate	Cut order: rainfall triggers, GEV fit, bond redemption, Blink, verify page

13 Limitations of the product

State these on stage briefly and in the README in full. Naming them first is more credible than being caught on them.

13.1 Insurance design

- **Basis risk.** Payouts follow a gauge, not your property. Flooding varies street by street with elevation and drainage, so people can be paid without loss or flooded without payment. This cannot be eliminated, only reduced.
- **Fixed benefit.** Atlas pays a set amount for immediate needs. It does not cover property damage and does not replace flood insurance. Users may misunderstand this unless it is stated clearly at purchase.
- **One payout per term.** Back-to-back floods in one term pay once.
- **Adverse selection.** Waiting periods and sales freezes reduce, but do not remove, buying ahead of known risk.

13.2 Data and oracle

- **Gauge failure.** Gauges can be damaged or go offline in the largest floods, exactly when they matter most.
- **Provisional data.** Real-time readings are provisional and can be revised later. A payout made on a provisional reading cannot be clawed back.
- **Single source.** The live feed depends on one public data provider. Production would need several independent sources.
- **Replay trust.** The demo replay feed reads the team's server, so the replay demonstrates the mechanism, not trustlessness.

- **Off-chain decision logic.** Without a custom program, the decision to pay is made by the team's bot. The oracle value is verifiable, but enforcement is not; a bot could fail to pay. A Solana program would make payout automatic and is the production step.

13.3 Pricing and capital

- **Few extreme events.** Decades of data contain only a handful of major floods per gauge, so premiums carry wide uncertainty.
- **Non-stationarity.** Development, drainage projects and climate change shift flood frequency. History may understate future risk.
- **Correlated losses.** One storm triggers most of the book at once. Beyond reserves plus bond capital, payouts are reduced pro rata.
- **Spending limit tension.** The bot's cap protects the pool from a compromised key, but in a catastrophe it slows payouts until multisig approval of a larger batch.
- **Bond investor risk.** Bond holders can lose most of their principal in one event. Pricing that risk properly needs far more modeling than a hackathon allows.

13.4 Legal and regulatory

- **Insurance licensing.** Selling Atlas policies to the public would require a licensed insurer or an approved program structure.
- **Securities.** The catastrophe bond token is likely a security, with registration or exemption requirements.
- **Consumer protection.** Disclosure of basis risk and terms would be regulated.
- **Stablecoin dependence.** Payouts depend on the stablecoin and its issuer.

13.5 Users and adoption

- **Wallet access.** Many of the people most exposed to flood losses do not hold stablecoins or a crypto wallet. Onboarding and off-ramps to bank accounts are unsolved in this build.
- **Key loss.** A lost wallet means a lost policy, since policies are non-transferable.
- **Trust.** People in a crisis may distrust an unfamiliar product.

13.6 Hackathon build

Devnet only, test money only, a small gauge set, shortened waiting periods in demo mode, custodial bot keys, and no audit of any code.

14 Submission checklist

- ☐ Repository public with README: problem, architecture diagram, setup, how to run the replay, limitations.

- ☐ Every on-chain address listed: policy mint example, vault, bond mint, feed accounts, sample payout transactions.
- ☐ Demo video (2 to 3 minutes) of the full Harvey replay.
- ☐ Slides: problem, demo, pricing, stress test, Tiger Data usage, Solana features used, limitations, next steps.
- ☐ Submitted to Tiger Data track, Solana track and overall, with each track's specific usage spelled out in the description.
- ☐ `/docs/numbers.sql` reproduces every number on the slides.

15 Unverified assumptions to check in Hour 0 to 1

1. USGS instantaneous and peak data endpoints, parameters, rate limits and history depth for the chosen gauges.
2. Harris County Flood Warning System public API availability.
3. NWS flood stage data access for the chosen forecast points.
4. Switchboard on-demand custom HTTP feeds created and updated on devnet from TypeScript.
5. Squads spending limits and time locks via SDK on devnet.
6. Token-2022 NonTransferable, metadata and PermanentDelegate combined on one mint via JS.
7. Current Solana Actions and Blinks specification and testing tools.
8. Tiger Cloud free-tier limits, and whether the Tiger Data track requires Tiger Cloud specifically.
9. Devnet USDC faucet availability.
10. Exact Harvey dates and gauge behavior in the stored data, rather than from memory, before quoting them on stage.